Anexo: estrategia de testing y calidad

Las pruebas se aplican a la funcionalidad de la prueba de concepto: **cambiar el estado de una orden de trabajo** (`PATCH /api/orders/:id/status`), que solo acepta las transiciones válidas del proceso de reparación. Los casos 1 y 2 **ya están implementados** en el repositorio (carpeta `api/test`) y pasan con `npm test`: en total la API tiene 87 pruebas y el front 82. El caso 3 está redactado para registrarse en Testiny, pero todavía no se cargó.

1. Prueba unitaria con Jest

Campo	Detalle
Qué se prueba	La función `canTransition(from, to)` de `api/src/domain/status.ts`, que decide si un cambio de estado es válido. Es una función pura: se prueba sola, sin servidor ni base.
Objetivo	Comprobar que el proceso no se puede saltear: una orden en diagnóstico no pasa directamente a lista para retirar.
Datos de entrada	Caso principal: `canTransition("diagnosing", "ready_for_pickup")`. Casos complementarios: `("diagnosing", "awaiting_approval")`, un estado hacia sí mismo y cualquier salida de `delivered` o `cancelled`.
Resultado esperado	`false` para el salto inválido. `true` para la transición válida. `false` para los estados finales.
Error que detecta	Que alguien edite la tabla de transiciones y deje pasar un salto inválido, por ejemplo entregar un vehículo sin presupuesto aprobado.
Dónde está	`api/test/domain/status.test.ts` `it("rejects skipping the process (diagnosing -> ready_for_pickup)")`

2. Prueba de integración con Supertest

Campo	Detalle
Método y ruta	`PATCH /api/orders/wo-001/status`
Datos enviados	Orden `wo-001` (código TR-1042, en diagnóstico). Encabezado `Authorization: Bearer <token>` obtenido con el login. Cuerpo: `{"status": "ready_for_pickup"}`.
Código esperado	**422** (transición inválida).
Respuesta esperada	`{ "error": "INVALID_TRANSITION", "message": "El cambio de estado no es válido para el estado actual de la orden.", "allowed": ["awaiting_approval", "cancelled"] }`
Qué verifica	Que la API aplica la regla en el servidor y no solo en la interfaz, informa qué estados sí son válidos y **no modifica la orden**: un `GET` posterior sigue devolviendo `diagnosing`. Otros casos de la misma ruta: sin token responde 401, estado inexistente 400 y orden inexistente 404.

Dónde está	`api/test/http/app.test.ts`
	`it("answers 422 with the allowed statuses for an invalid transition")`
	`it("does not modify the order after an invalid transition")`

3. Prueba manual (caso para Testiny)

Campo	Detalle
Identificador y nombre	**TC-001**: Un empleado cambia el estado de una orden y el cliente lo ve en el seguimiento.
Precondiciones	La API y el front están corriendo con los datos de ejemplo recién cargados (reiniciar la API los restablece). Se conoce el usuario y la contraseña del taller. La orden TR-1042 está en *En diagnóstico*.
Pasos	1. Abrir `/seguimiento?codigo=TR-1042` y ver el estado y el paso actual. 2. Ir a `/login` e ingresar con el usuario del taller. 3. Abrir la orden TR-1042 desde el tablero. 4. Mirar qué botones de cambio de estado aparecen. 5. Elegir *Pasar a Esperando aprobación del cliente*. 6. Volver a `/seguimiento?codigo=TR-1042` y recargar.
Resultado esperado	Paso 1: *En diagnóstico*, paso 2 de 6. Paso 4: solo *Esperando aprobación del cliente* y *Cancelado*, ningún otro. Paso 5: aviso de éxito, nuevo estado y una entrada nueva en el historial. Paso 6: *Esperando aprobación*, paso 3 de 6, con la patente parcialmente oculta y **sin nombre ni teléfono** del cliente.

4. Verificación y validación

	Pregunta que responde	Ejemplo con el escenario de Taller Ruiz
Verificación	¿Construimos el sistema **correctamente**, de acuerdo con su especificación?	Cada escenario de la especificación (por ejemplo "una orden en diagnóstico no puede pasar a lista para retirar") se contrasta con una prueba que lo ejecuta, y se revisa el código contra esa especificación. Resultado medible: las pruebas pasan y no quedan escenarios sin cubrir.
Validación	¿Construimos el sistema **correcto**, el que resuelve la necesidad real del cliente?	Una prueba piloto de dos semanas en el taller, con órdenes reales: ¿los clientes miran el seguimiento en lugar de llamar? ¿un empleado cambia un estado en menos de 30 segundos sin ayuda? ¿tres clientes entienden en qué etapa está su vehículo sin que se les explique? Se compara la cantidad de llamadas de consulta antes y después. *Se propone para el sistema completo: no se realizó.*

5. Por qué estas pruebas y qué riesgos reducen

Prueba	Por qué se eligió	Riesgo que reduce
Unitaria	La regla de transiciones es el corazón del negocio y se puede probar aislada, rápido y en cada cambio del código.	Saltear etapas del proceso, por ejemplo entregar sin presupuesto aprobado.
Integración	Comprueba la regla de punta a punta: autenticación, validación del cuerpo, código HTTP y que el estado no cambie tras un error.	Que la regla exista en el dominio pero la API no la haga cumplir, o que filtre datos o detalles internos.
Manual	Recorre lo que cliente y empleado usan de verdad, cruzando el front y la API, y mira cosas que una prueba automática no ve: textos, claridad y datos visibles.	Mostrar datos personales en el seguimiento público, o una interfaz que ofrezca acciones que la API rechaza.